

Introduction to Forward Deployed Engineering & Generative AI Foundations

Coder Army

Introduction to Forward Deployed Engineering

From a Business Problem to a Real Engineering Solution

Consider a large e-commerce company receiving **50,000 customer-support requests every day**.

Customers may ask:

- "Where is my order?"
- "I received the wrong product."
- "Can I return an item after 15 days?"
- "My payment was deducted, but the order was not created."
- "Please cancel my order and initiate a refund."
- "The delivery agent marked my order as delivered, but I never received it."

The company may initially describe its requirement as:

┆ "We want an AI chatbot that can answer customer questions."

This sounds like a requirement, but from an engineering perspective, it tells us very little.

The first thing we need to understand is:

┆ **What problem is the company actually trying to solve?**

Finding the Real Requirement

The company does not necessarily care about having a chatbot.

It cares about solving problems such as:

- Customers waiting too long for responses
- Support executives repeatedly answering the same questions
- Different executives giving inconsistent answers
- Support costs increasing as the company grows
- Important cases getting buried under routine queries
- Customers contacting multiple departments for one problem
- Support being unavailable outside working hours

Therefore, a better requirement would be:

Reduce the time required to resolve customer problems without reducing accuracy or customer trust.

This is very different from simply saying:

"Build an AI chatbot."

The chatbot is only one possible solution.

Attempt 1: Rule-Based Customer Support

Before using AI models, we could build a traditional rule-based chatbot.

For example:

```
IF message contains "order status"  
THEN show order tracking page  
  
IF message contains "refund"  
THEN show refund policy
```

```
IF message contains "cancel"  
THEN show cancellation instructions
```

This works well when inputs are predictable.

The problem is that humans do not communicate through fixed commands.

A customer may say:

```
Where is my order?
```

Another customer may say:

```
It has been seven days and my package has not arrived.
```

Another may write:

```
Bhai mera parcel abhi tak nahi aaya.
```

And another may say:

```
The expected delivery date has passed. Could you check what happened?
```

All four customers have roughly the same intention.

Their words, however, are completely different.

Why not simply add more rules?

We could keep creating more conditions, but the system would quickly become difficult to manage.

Problems include:

- Thousands of possible sentence variations
- Rules conflicting with each other
- New phrasing requiring new conditions

- Spelling mistakes
- Multiple languages
- Hinglish
- Context from previous messages
- Multiple problems inside one message
- Unexpected situations

The fundamental issue is:

Human language has too many variations to completely describe using handwritten rules.

Attempt 2: Machine Learning Classification

Instead of manually creating every rule, we could train a machine-learning model.

We provide examples such as:

Customer Message	Category
Where is my package?	Order Status
Cancel my purchase	Cancellation
When will I get my money back?	Refund
The product arrived broken	Damaged Product

The model learns patterns from these examples.

Now the flow becomes:

```

Customer Message
  ↓
Machine Learning Model
  ↓
Predicted Category
  
```

This is better than keyword matching.

But it still solves only a small part of the actual customer-support problem.

A model may correctly identify:

```
Category = Refund
```

But that does not mean it can automatically:

- Explain the company's refund policy
- Find the customer's order
- Check refund eligibility
- Ask for missing information
- Start the refund process
- Handle unusual cases
- Escalate to a human
- Explain why a request was approved or rejected

Classification solves one part of the workflow.

It does not solve the complete business problem.

Attempt 3: Using a General-Purpose LLM

Large Language Models are much more flexible.

An LLM can potentially:

- Understand a customer's request
- Summarize a complaint
- Detect intent
- Extract information such as an order ID
- Translate text
- Generate responses

- Ask follow-up questions
- Read supplied company policies
- Decide which available tool may be useful

For example:

System:
You are a customer-support assistant.
Answer the customer's question.

Customer:
Can I return a laptop after 20 days?

The model might respond:

Yes, laptops can be returned within 30 days of delivery.

The response sounds professional and confident.

But suppose the company's actual policy allows laptop returns for only **7 days**.

The answer is now completely wrong.

This gives us an important engineering lesson:

Generating a convincing answer and generating a correct business answer are different problems.

An LLM Does Not Automatically Know Your Company

A general-purpose LLM may understand what concepts such as refunds, cancellations, payments and deliveries mean.

But it does not automatically know:

- Your company's current refund policy
- A customer's order history

- Current delivery status
- Payment database records
- Inventory information
- Internal business procedures
- Recent policy changes
- Customer identity
- Customer permissions

The application therefore needs to provide the model with relevant information.

Conceptually:

```

Customer Question
  +
Relevant Company Policy
  +
Current Order Details
  +
Previous Conversation
  ↓
  LLM
  ↓
Grounded Response
  
```

The LLM is only one component of the application.

Generating Text Is Not the Same as Performing an Action

Suppose a customer says:

| "Cancel my order and refund my payment."

An LLM can easily generate:

```

Your order has been cancelled.
  
```

But generating this sentence does not actually cancel anything.

A real system needs something closer to:

```
graph TD; A[Customer Request] --> B[LLM Understands Required Action]; B --> C[Application Validates Request]; C --> D[Cancellation API Called]; D --> E[Database Updated]; E --> F[Confirmed Result Returned];
```

Customer Request
↓
LLM Understands Required Action
↓
Application Validates Request
↓
Cancellation API Called
↓
Database Updated
↓
Confirmed Result Returned

Once we start building this system, many additional engineering questions appear:

- Is this user authorized to cancel the order?
- Has the product already been shipped?
- Is this particular order cancellable?
- Should an LLM directly call the cancellation service?
- Should large refunds require human approval?
- What happens if the API fails?
- What if the model extracts the wrong order ID?
- How do we audit everything that happened?
- How do we test the system?
- How much does each conversation cost?
- What latency is acceptable?
- When should the system escalate to a human?

Our original requirement:

“Build an AI chatbot.”

has now become:

Build a secure and reliable system that combines an AI model, company knowledge, customer data, existing APIs, validation rules and human oversight to resolve customer-support requests.

The difference between these two statements is where **Forward Deployed Engineering** becomes important.

What Is a Forward Deployed Engineer?

The term can be understood by breaking it into parts.

What Does "Deployed" Mean?

Here, deployed does not simply mean:

| Deploy an application to a server.

It means placing an engineer close to the environment where the real problem exists.

A Forward Deployed Engineer works closely with:

- Customers
- Clients
- Business departments
- Operations teams
- Existing engineering teams

The engineer observes the real workflow, understands the constraints, builds a solution and helps put that solution into actual production use.

Why "Forward"?

A traditional product team generally builds capabilities that can serve many customers.

Product Engineering Team
↓
General Platform
↓
Many Customers

A Forward Deployed Engineer works much closer to a specific customer's problem.

Customer's Real Problem
↑
Forward Deployed Engineer
↑
Product + Data + APIs +
Engineering Teams

The FDE operates at the boundary between:

Technology and the customer's actual work.

Forward Deployed Engineer — Definition

A Forward Deployed Engineer is a software engineer who works closely with customers or operational teams to understand important problems, build technical solutions around their real systems, and take those solutions into production.

The role combines multiple engineering skills:

- Software engineering
- Product thinking
- System design
- Integrations
- Rapid experimentation
- Production delivery

- Evaluation
- Troubleshooting

One important difference is that the customer may not know exactly what technical solution they need.

Discovering the correct solution is itself part of the FDE's job.

Request vs Requirement

A customer may say:

| "We need an AI chatbot."

That is usually a **proposed solution**.

The engineer needs to discover the actual **requirement** behind it.

For example:

```
Customer Request
"We need an AI chatbot."
↓
Understand Business Problem
↓
Understand Existing Workflow
↓
Identify Constraints
↓
Design Technical Solution
↓
Integrate With Existing Systems
↓
Test and Evaluate
↓
Production
```

That is the mindset we will use throughout this series.

Goal of This Series

By the end of this series, the goal is to be able to take a vague business request such as:

“Build an AI assistant for our support department.”

and convert it into a real engineering solution using **Generative AI capabilities**.

Before doing that, however, we need to understand the technology we are going to build around.

So we first need to understand **AI itself**.

Part 2 — Understanding Artificial Intelligence

Where Did AI Begin?

The fundamental question behind Artificial Intelligence is simple:

Can a machine perform tasks that appear to require human intelligence?

Early computers were primarily designed to perform calculations and follow explicit instructions.

For example:

```
Input: 20 + 30  
Instruction: Add the numbers  
Output: 50
```

Programming languages gave us constructs such as:

- Variables
- Conditions
- Loops
- Functions

- Recursion
- Data structures
- Algorithms

All of these allow humans to describe instructions that a computer can execute.

The important thing is that traditional programs work extremely well when:

- The input is clear
- The operation can be precisely defined
- The expected result is known
- Every step can be expressed as an instruction

Human intelligence is much more complicated.

Humans can:

- Understand language
- Recognize objects
- Solve unfamiliar problems
- Learn from experience
- Make decisions with incomplete information
- Explain ideas
- Create stories
- Create images
- Create music
- Write programs

This leads to a much deeper question:

| Can intelligence itself be described as a computational process?

Alan Turing and the Question of Machine Intelligence

In 1950, Alan Turing published *Computing Machinery and Intelligence*.

The paper began with a famous question:

Can machines think?

The problem is that even for humans, “thinking” is difficult to define precisely.

We cannot directly observe someone’s thoughts.

We observe behaviour and infer intelligence.

Turing therefore shifted the discussion from:

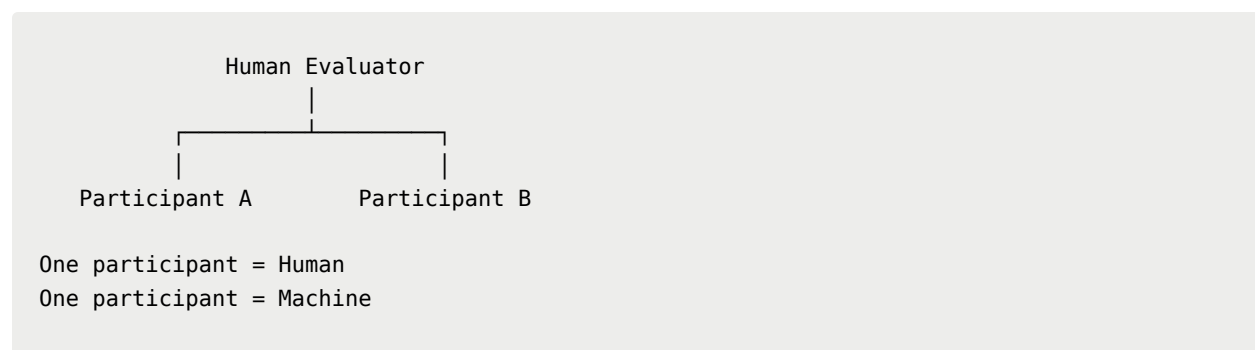
Does the machine genuinely think?

toward something more observable:

Can a machine communicate convincingly enough that a human evaluator cannot reliably distinguish it from another human?

This idea later became popularly known as the **Turing Test**.

A simplified setup looks like:



Communication happens through text.

The evaluator does not know which participant is the machine.

The interesting idea here was not that the Turing Test explained how to build AI.

Instead, it shifted the discussion from an abstract philosophical question toward **observable intelligent behaviour**.

If a machine can:

- Understand questions
- Maintain context
- Provide relevant answers
- Explain ideas
- Respond to unfamiliar situations
- Communicate naturally

then it displays behaviours associated with intelligence.

However:

| **Sounding intelligent does not automatically mean being correct.**

This distinction becomes extremely important with modern LLMs.

The Dartmouth Workshop and the Term "Artificial Intelligence"

In 1955, researchers proposed a workshop that was held at Dartmouth College in 1956.

The proposal used the term:

| **Artificial Intelligence**

This helped establish AI as a distinct area of research.

The broad idea was:

| If aspects of intelligence can be described precisely enough, perhaps machines can be made to simulate them.

Researchers explored areas such as:

- Language
- Problem solving
- Forming abstractions
- Learning from experience
- Tasks associated with human intelligence

The dream of AI is therefore not new.

What changed dramatically over the following decades was our ability to build increasingly powerful systems using better algorithms, more data and much greater computational power.

AI Is Much Bigger Than LLMs

Artificial Intelligence is a broad field.

Examples of AI systems include:

- Chess-playing programs
- Medical diagnosis systems
- Face-recognition systems
- Recommendation engines
- Self-driving systems
- Speech-recognition systems
- Language models
- Image-generation models

Therefore:

Artificial Intelligence
 $\neq$
 Large Language Models

An LLM is one kind of AI system.

Generative AI is one major area inside the much larger field of Artificial Intelligence.

Rule-Based AI

One early approach to AI was based on a simple idea:

If intelligent behaviour follows rules, perhaps humans can write those rules explicitly.

Suppose we want to decide whether someone is eligible to vote.

```
IF age >= 18
THEN eligible
ELSE not eligible
```

The logic is deterministic.

```
Age 17 → Not Eligible
Age 18 → Eligible
Age 25 → Eligible
```

We could also create a troubleshooting rule:

```
IF device does not turn on
AND battery level = 0
THEN recommend charging the device
```

In these systems, humans provide the knowledge explicitly.

```
Human Expert Writes Rules
      ↓
Computer Applies Rules
      ↓
Conclusion / Action
```

This approach is commonly associated with **symbolic AI**.

Components of a Rule-Based System

A simple rule-based system generally contains three important ideas.

1. Facts

Facts represent the current situation.

```
Customer has a premium account.  
Order was delivered 3 days ago.  
Product is returnable.  
Product is damaged.
```

2. Rules

Rules determine what should happen.

```
IF product is damaged  
AND delivery was within 7 days  
AND product is returnable  
THEN customer is eligible for replacement
```

3. Inference

The system combines the available facts with the rules to reach a result.

The computer is not inventing the policy.

It is applying logic created by humans.

The Natural Language Problem

The business rule itself may be simple.

For example:

```
IF daysSinceDelivery <= 7
THEN returnEligible = true
```

But customers do not usually send structured input such as:

```
{
  "daysSinceDelivery": 4
}
```

They may instead say:

“I received this sometime last week. Can I send it back?”

Now several problems appear.

The system must understand:

- What does “this” refer to?
- What date does “last week” represent?
- Which order is the customer talking about?
- Is the customer asking for a refund or replacement?
- Is the product returnable?

The business rule may be easy.

Converting natural human language into the facts required by the rule is difficult.

Early Chatbots and Expert Systems

Early AI research explored multiple directions.

Two useful categories are:

```
Chatbots
→ Can a machine communicate like a person?
```

Expert Systems

→ Can a machine make decisions like a specialist?

These represented different aspects of what humans considered intelligent behaviour.

ELIZA — An Early Chatbot

ELIZA was one of the early systems designed to simulate conversation.

A conversation could look like:

```
User:
I am feeling unhappy.

ELIZA:
Why are you feeling unhappy?

User:
My friends do not understand me.

ELIZA:
Why do you think your friends do not understand you?

User:
They never listen to me.

ELIZA:
Can you tell me more about them?
```

The responses could feel surprisingly meaningful.

But the system was not deeply understanding human emotions.

It largely worked through **patterns and response templates**.

For example:

```
IF user says:
"I am <something>"

RESPOND:
"Why are you <something>?"
```

Another rule might be:

```
IF user mentions "mother"  
  
RESPOND:  
"Tell me more about your family."
```

Pronouns and sentence patterns could also be transformed to create responses that looked conversational.

The Illusion of Understanding

ELIZA demonstrated an important property of human interaction:

A system can produce language that feels meaningful without deeply understanding the situation.

Humans naturally assign meaning and intention to language.

If a computer asks:

"How does that make you feel?"

we may assume that the system understands exactly what **"that"** refers to.

But the response may simply have been selected from a template.

Modern LLMs are vastly more capable than early chatbots and are not simply collections of these templates.

However, one important lesson remains:

Language fluency should not be confused with factual reliability.

A fluent system may cause users to assume that:

- The system knows whether something is true
- It understands the world exactly like humans do
- It verified the information before answering
- It remembers information that was never provided

- It performed an action that it only described

These assumptions can be dangerous when building real applications.

Why Handwritten Rules Do Not Scale

Imagine a customer-support system needs to understand only ten intentions:

```
Order Tracking
Cancellation
Refund
Replacement
Payment Failure
Damaged Product
Missing Item
Account Problem
Delivery Complaint
Product Information
```

Now consider only **Order Tracking**.

Customers could say:

```
Where is my order?

Track my package.

My parcel has not arrived.

Delivery kab hogi?

Order abhi tak nahi mila.

The expected date has passed.

Can you check shipment ORD-123?

Your app says dispatched, but nothing arrived.
```

Now add:

- Typing mistakes
- Abbreviations

- Sarcasm
- Different languages
- Hinglish
- Missing information
- Multiple issues in one message
- Previous conversation context
- Different policies
- Regional differences
- Policy changes

The number of combinations becomes enormous.

This led researchers toward another idea:

Instead of describing every rule ourselves, can the machine **learn useful patterns from examples?**

Machine Learning

Learning Patterns Instead of Writing Every Rule

Imagine teaching a child to recognize a dog.

We could try to define rules:

```
A dog has four legs.
A dog has fur.
A dog has two ears.
A dog has a tail.
```

Unfortunately, these rules also describe many other animals.

We could continue adding more rules:

A dog usually barks.
A dog has a particular face.
A dog has particular body proportions.

But dog breeds themselves vary significantly.

A Chihuahua and a Great Dane look very different.

Humans usually learn this concept through examples:

This is a dog.

This is also a dog.

This is a cat.

This is another dog.

After seeing enough examples, we develop an internal ability to distinguish dogs without receiving a perfect mathematical definition.

Machine learning follows a related principle:

Provide examples and allow the system to learn patterns that map inputs to outputs.

Traditional Programming vs Machine Learning

Traditional programming:

Input
+
Human-Written Rules
↓
Output

Machine-learning training:


```

Training Inputs
  +
Correct Outputs
  ↓
Learning Algorithm
  ↓
Model
  
```

Using the trained model:

```

New Input
  +
Trained Model
  ↓
Predicted Output
  
```

This represents an important shift.

In normal programming, humans explicitly provide the rules.

In machine learning, humans generally provide:

- Data
- Examples
- Learning objectives
- Model architecture
- Training process

The model adjusts its internal parameters to discover useful patterns.

Example: Spam Detection

A rule-based spam filter might use:

```

IF email contains "win money"
THEN spam
  
```

But this immediately creates problems.

A legitimate email may contain the words “win money”.

A spammer may simply change their wording.

Instead, a machine-learning system can learn patterns from many examples of:

```
Email → Spam  
Email → Not Spam
```

The model learns statistical relationships from the data rather than relying only on one manually written rule.

Unsupervised Learning

Not every machine-learning problem comes with labelled examples.

Suppose we have one million customer messages, but nobody has categorized them.

A system might discover clusters such as:

```
Group 1 → Delivery-related conversations  
Group 2 → Payment-related conversations  
Group 3 → Product-quality complaints
```

The system discovers structure within the available data without being explicitly told the correct category for each individual example.

This is broadly known as **unsupervised learning**.

Machine Learning Does Not Automatically Learn Truth

A model learns patterns from the data it receives.

Therefore, if the data is:

- Incorrect
- Incomplete
- Biased
- Outdated
- Unrepresentative

the model can learn and reproduce those problems.

Machine learning is not a magical mechanism that automatically discovers objective truth.

What Is a Model?

The word **model** appears everywhere in AI.

At the simplest level:

A model is a computational system that maps an input to an output.

```

Input
↓
Model
↓
Output
  
```

Examples:

```

Email
↓
Spam Detection Model
↓
Spam Probability
  
```

```

Photograph
↓
Image Model
  
```

↓
Objects Detected

For a language model:

Previous Text
↓
Language Model
↓
Probabilities for the Next Token

Training

During training:

- The model receives many examples
- It generates outputs
- Those outputs are evaluated according to a training objective
- Internal parameters are adjusted
- The process is repeated many times

Conceptually:

Training Data
+
Training Process
↓
Parameters Adjust
↓
Trained Model

Large modern models may contain billions of learned parameters.

Training these models can require enormous amounts of computation.

Inference

Once a model is trained, we can use it on new inputs.

This is called **inference**.

```
New Input
+
Existing Model Parameters
↓
Output
```

For an LLM:

```
Prompt
+
Existing Model
↓
Generated Response
```

When an application calls an LLM through Spring AI, Python, JavaScript or an API, it is normally performing **inference**.

Does the Model Learn From Every Prompt?

Normally, a prompt is temporary input.

```
Prompt
↓
Model
↓
Response
```

The prompt influences the current response.

But a normal inference request does not directly modify the model's underlying trained parameters.

Therefore:

Providing information in a prompt and training the model are two different things.

From Machine Learning to Neural Networks

Traditional machine-learning systems often relied heavily on something called **feature engineering**.

Suppose we want a computer to recognize faces.

A computer does not initially see:

```
Person
Eyes
Nose
Mouth
```

It sees numerical pixel values.

Humans might therefore design features such as:

- Edges
- Shapes
- Colour differences
- Eye-like patterns
- Symmetry
- Distances between facial components

For text, humans might design features such as:

- Word frequency
- Keywords
- Sentence length
- Punctuation
- Grammatical tags

The process of manually deciding which characteristics the model should examine is called:

Why Feature Engineering Became a Bottleneck

Feature engineering presented several problems.

1. It required significant human effort

Engineers had to design useful features for individual problems.

2. Humans might miss important patterns

The most useful patterns may be difficult for humans to identify manually.

3. Features often did not transfer well

Features designed for face recognition may be useless for:

- Dog recognition
- Speech recognition
- Language understanding

This raises an important question:

What if the useful pattern is too complicated for a human to describe manually?

Neural networks provide one way of approaching this problem.

Building the Idea of a Neuron

Suppose we want to predict whether a customer-support ticket is urgent.

We could use several inputs:

```
x1 = Contains the word "urgent"  
x2 = Payment failed
```

x_3 = Customer contacted support multiple times
 x_4 = Customer is locked out

Not every input should matter equally.

For example, being completely locked out of an account might matter more than simply containing the word “urgent”.

A model can assign a numerical **weight** to each input.

Conceptually:

```
Inputs
  ↓
Weighted Combination
  ↓
Activation
  ↓
Output
```

This simplified computational unit is inspired by the idea of a biological neuron.

A single unit can model only limited relationships.

Many units can be connected together.

Neural Networks

A neural network consists of layers of connected computational units.

```
Input Layer
  ↓
Hidden Layer
  ↓
Hidden Layer
  ↓
Output Layer
```

During training, the network adjusts large numbers of weights.

This allows the system to learn complicated relationships between the input and the expected output.

Representation Learning

One of the major strengths of deep learning is **representation learning**.

Older machine-learning approaches often depended heavily on humans deciding how information should be represented.

Deep-learning models can learn useful internal representations directly from data.

For example:

```
Where is my parcel?  
  
Track my package.  
  
My delivery has not arrived.  
  
Order abhi tak nahi aaya.
```

These sentences use different words.

But they communicate closely related ideas.

A neural network can learn representations that capture relationships between them.

This does not mean the network contains something simple such as:

```
ORDER_TRACKING = these exact sentences
```

Instead, information is distributed across many learned numerical parameters and representations.

AI, Machine Learning and Deep Learning

These concepts are related rather than completely separate.

```
Artificial Intelligence  
|  
└─ Machine Learning
```

Our goal in this series is not to become deep-learning researchers.

We need enough understanding of these systems to build **reliable applications around them**.

Neural Networks and Computer Vision

Neural networks achieved major success in areas such as computer vision.

Convolutional Neural Networks, or **CNNs**, became particularly important for image-related problems.

They enabled systems to learn visual patterns such as:

- Edges
- Shapes
- Textures
- Objects
- Faces

Applications include:

- Facial recognition
- Photo tagging
- Object detection
- Autonomous driving systems

Instead of manually specifying every visual feature, deep-learning systems could learn increasingly useful representations from training data.

The Language Problem

Language presented a different set of challenges.

Earlier neural language systems commonly used architectures such as:

- RNNs
- LSTMs

Before understanding their limitations, we need to understand why language itself is such a difficult problem.

Why Human Language Is Difficult for Computers

Language feels natural to humans because we spend our entire lives learning and using it.

For machines, the meaning of a sentence is rarely determined by individual words alone.

Meaning depends on:

- Context
 - Word order
 - Grammar
 - Speaker intention
 - Shared knowledge
 - Culture
 - Previous conversation
 - Tone
 - Ambiguity
-

Same Word, Different Meaning

Consider the word:

bank

It could refer to:

A financial institution

or:

The side of a river

The individual word alone is not enough.

Context determines the likely meaning.

Different Words, Similar Meaning

Where is my order?

and:

Track my package.

contain different words but may represent almost the same intention.

A language system therefore needs to recognize meaning beyond exact word matching.

Word Order Matters

Consider:

The dog chased the man.

versus:

The man chased the dog.

Almost all the words are identical.

But changing their order completely changes the meaning.

Negation Matters

The customer received the refund.

is very different from:

The customer did not receive the refund.

One word changes the meaning of the entire sentence.

References Depend on Context

Consider:

Riya placed the laptop on the table because it was unstable.

What does `it` refer to?

- Laptop?
- Table?

Now consider:

Riya placed the laptop on the table because it was heavy.

We are more likely to interpret `it` as the laptop.

Resolving such references requires context.

Sarcasm and Tone

Consider:

Amazing service. My order is only three weeks late.

A simple keyword-based system may see:

amazing
service

and classify the statement as positive.

A human immediately recognizes the sarcasm.

Language Is Productive

Humans constantly create sentences that have never been spoken before.

Therefore, a useful language system cannot simply memorize every possible sentence.

It needs to learn patterns that generalize to new combinations of words and ideas.

The Evolution of Language Models

A **language model** tries to estimate which pieces of language are likely in a particular context.

Consider:

The sun rises in the ____

Possible continuations include:

east
morning
sky

In this exact context, **east** is highly likely.

Now consider:

She deposited the money in the ____

Here:

bank

becomes highly likely.

The important idea is:

The likelihood of the next piece of text depends on the context that came before it.

Early Statistical Language Models

Before modern neural networks, one approach was simply to count how frequently words appeared together.

Suppose the training text contains:

I like machine learning.
I like Java programming.
Students like machine learning.

After the word:

```
like
```

we observe:

```
machine → 2 times  
Java    → 1 time
```

We could estimate:

```
P(machine | like) = 2/3  
P(Java | like) = 1/3
```

Therefore, if the model sees:

```
I like ____
```

`machine` receives a higher probability than `Java`.

This is a basic statistical language model.

N-Grams

An **n-gram** is a sequence of `n` items, commonly words or characters.

Unigram

One item:

```
machine
```

Bigram

Two items:

```
machine learning
```

Trigram

Three items:

```
I like Java
```

An n-gram language model predicts upcoming text using a limited amount of previous context.

A bigram model may use:

```
P(next word | previous word)
```

A trigram model may use:

```
P(next word | previous two words)
```

Why Context Matters

Suppose the sentence is:

```
I went to the bank to deposit some ____
```

Possible next words include:

```
money  
cash  
funds
```

If the model only considers:

some

it has very little information.

Compare the available context:

bank

the bank

went to the bank

went to the bank to deposit some

As more context becomes available, the likely continuation becomes easier to estimate.

This gives us one of the most important principles in language modelling:

The meaning and likely continuation of text depend heavily on context.

Problems With N-Gram Models

Suppose a model learns from:

The customer requested a refund.

but never observes:

The buyer asked for reimbursement.

A simple count-based system may treat these as completely unrelated sequences.

Humans immediately recognize that:

```
customer ≈ buyer  
requested ≈ asked for  
refund ≈ reimbursement
```

Language also contains an enormous number of possible word combinations.

Most possible sequences occur rarely or may never appear in the available training data.

This is known as the **sparsity problem**.

Modern language models needed a better way to learn relationships between words and context.

Neural Network-Based Language Models

Neural networks provided a way to learn richer representations of language.

Earlier neural approaches frequently used **Recurrent Neural Networks**, or RNNs.

Recurrent Neural Networks — Basic Intuition

An RNN processes language sequentially.

Conceptually:

```
Read Word  
↓  
Update Internal State  
↓  
Read Next Word  
↓
```

Update Internal State
↓
Continue...

Consider:

The payment has not been refunded.

An RNN might process it conceptually as:

Initial State
↓
"The"
↓
State 1
↓
"payment"
↓
State 2
↓
"has"
↓
State 3
↓
"not"
↓
State 4
↓
"been"
↓
State 5
↓
"refunded"
↓
Final State

The internal state tries to preserve useful information about the words processed earlier.

Why Sequential Processing Makes Sense

Human language itself arrives sequentially.

We hear:

```
The → customer → requested → a → refund
```

one word after another.

Sequential processing also naturally preserves word order.

The problem appears when the sequence becomes long.

The Information Bottleneck

An RNN repeatedly compresses previously seen information into an evolving internal state.

Imagine reading a 20-page policy one word at a time while continuously maintaining a small set of notes containing everything important.

As the sequence grows:

- Earlier information may weaken
- Important details may be lost
- Irrelevant information consumes capacity
- Distant relationships become harder to preserve

Consider:

```
The replacement policy for laptops purchased during the festival sale,  
which was updated after several suppliers changed their warranty terms,  
does not apply to refurbished devices.
```

To properly understand:

```
does not apply
```

the model needs to connect it with:

The replacement policy

while also understanding that the exclusion concerns:

refurbished devices

Many unrelated words appear between these pieces of information.

Long-Range Dependencies

A **long-range dependency** occurs when interpreting one part of a sequence requires information from a distant part.

For example:

The laptop that the customer ordered after comparing several models from different brands was damaged.

What was damaged?

The laptop

The model must preserve that relationship across many intermediate words.

Handling these long-range relationships efficiently became an important challenge in language modelling.

Attention

Attention can be understood by starting with the problem it solves.

Consider:

Riya gave Priya the laptop because she needed it for work.

To understand:

she

the model needs to determine whether it refers to Riya or Priya.

To understand:

it

the model needs to connect it with:

laptop

Different words need information from different parts of the sentence.

The model therefore needs a mechanism that allows one position to examine other relevant positions.

That is the basic intuition behind **attention**.

Attention as Relevance

When processing a piece of language, the model can conceptually ask:

Which other parts of the context are useful for understanding this part right now?

Consider:

The animal did not cross the street because it was tired.

When processing:

it

the word:

animal

is probably more relevant than:

street

because an animal can be tired while a street normally cannot.

Conceptually:

"it" attending to:

animal	→ high relevance
cross	→ some relevance
street	→ lower relevance
because	→ structural relevance
tired	→ high relevance

These relationships are not manually programmed using rules such as:

```
IF word = "it"  
THEN look for animal
```

The useful attention patterns are learned through training.

Attention Is Context-Dependent

Consider:

I deposited money in the bank.

For the word `bank`, relevant surrounding information includes:

```
deposited  
money
```

Now consider:

```
We sat on the bank of the river.
```

Relevant context includes:

```
sat  
river
```

The word is the same.

Its meaning changes because the surrounding context changes.

Attention provides a mechanism for combining information from relevant parts of that context.

The Transformer Breakthrough

Transformers use attention as a central mechanism for processing sequences.

This architecture addressed several important limitations of older sequential models and became the foundation of modern Large Language Models.

It enabled models to work with relationships between different parts of a sequence much more effectively.

Modern systems such as ChatGPT, Gemini, Grok and many other language models are built around Transformer-based architectures.

Attention Is Not Intelligence by Itself

Attention is ultimately a mathematical mechanism for determining and combining relevant information.

By itself, attention does not guarantee that a model:

- Knows whether a statement is true
- Understands something exactly as a human does
- Has access to current information
- Has access to private company data
- Can perform real-world actions
- Will always make the correct decision

This distinction becomes extremely important when engineering applications around LLMs.

Welcome to Modern Large Language Models

Modern LLMs combine:

- Large neural networks
- Huge amounts of training data
- Learned numerical representations
- Transformer architectures
- Attention mechanisms
- Large-scale computational resources

One of the most recognizable names in this space is:

GPT

GPT stands for:

G → Generative
P → Pre-trained
T → Transformer

Each part tells us something important about how the model works.

G — Generative

A generative model creates new output.

For a GPT-style language model, generation can be understood as continuing a sequence.

Example:

Input:

Artificial intelligence can help developers

Possible generated continuation:

build applications that understand and produce language.

A generative language model can produce:

- Answers
- Summaries
- Explanations
- Stories
- Emails
- Code
- Structured data
- Tool-call requests

At the foundation of this generation process is a surprisingly simple idea:

Repeated next-token prediction.

The model predicts what token is likely to come next, adds a token, and repeats the process.

We will study exactly how this works in the next part.

P — Pre-Trained

Modern language models are trained on large amounts of data before developers begin building applications with them.

Conceptually:

```
graph TD; A[Large-Scale Pretraining] --> B[Reusable General Model]; B --> C[Many Different Applications];
```

The model does not start learning language from scratch whenever a developer creates an application.

The same pretrained model can potentially be used for:

- Customer support
- Coding
- Document analysis
- Education
- Content generation
- Information extraction
- Search-related workflows
- Data processing

Application developers provide additional instructions and context to guide the model toward a particular task.

T — Transformer

The **T** in GPT stands for:

| Transformer

GPT-style models use Transformer architecture, particularly a decoder-style Transformer for autoregressive language generation.

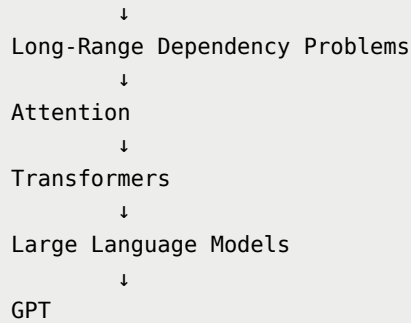
The Transformer gives the model mechanisms for processing context and understanding relationships between different parts of the input.

This architecture is one of the major reasons modern language models can handle language much more effectively than earlier approaches.

Complete Journey So Far

The evolution we have followed can be summarized as:

```
Traditional Programming
  ↓
Explicit Rules
  ↓
Rule-Based / Symbolic AI
  ↓
Machine Learning
  ↓
Learn Patterns From Data
  ↓
Neural Networks
  ↓
Deep Learning
  ↓
Representation Learning
  ↓
Statistical Language Models / N-Grams
  ↓
Neural Language Models
  ↓
RNNs / Sequential Processing
```



The Bigger Picture

The important lesson is that modern Generative AI did not appear suddenly. Each generation of technology tried to solve a limitation of the previous approach.

Rule-Based Systems

Humans explicitly wrote the knowledge.

Problem: Rules became impossible to maintain for complex environments such as natural language.

Machine Learning

Models learned patterns from examples.

Improvement: We no longer needed to manually define every rule.

Neural Networks and Deep Learning

Models learned increasingly complex representations from large amounts of data.

Improvement: Reduced dependence on manual feature engineering.

Early Language Models

Statistical systems estimated which words were likely to appear together.

Problem: Limited context and sparsity.

RNNs

Neural networks could process sequences while carrying information forward.

Problem: Long sequences and distant relationships remained difficult.

Attention and Transformers

Different parts of a sequence could dynamically use information from other relevant parts.

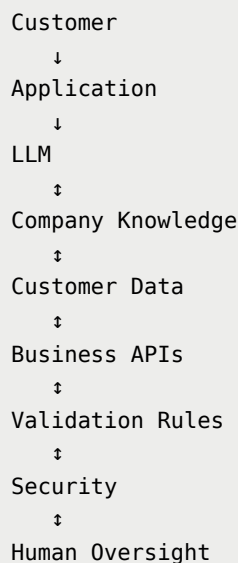
Result: The foundation for modern Large Language Models.

Connection Back to Forward Deployed Engineering

Understanding how an LLM works is important because an FDE should not treat the model as magic.

An LLM is a powerful component, but a real production system still needs engineering around it.

Returning to our original support example:



The LLM helps us work with language and reasoning-like tasks.

The application around the LLM ensures that the final system is:

- Useful

- Grounded
- Secure
- Reliable
- Testable
- Observable
- Connected to real systems
- Capable of performing real actions

This is the engineering problem we will learn to solve throughout the **Forward Deployed Engineering** series.

Key Takeaways

1. A customer's proposed solution is not always the real requirement.
2. Forward Deployed Engineers work close to real customer problems and convert business needs into production-ready technical solutions.
3. Artificial Intelligence is a much broader field than Generative AI or LLMs.
4. Rule-based systems depend on humans explicitly defining knowledge and logic.
5. Human language is too varied and contextual to handle completely through handwritten rules.
6. Machine learning allows systems to learn useful patterns from examples.
7. A model is fundamentally a computational system that maps inputs to outputs using learned parameters.
8. Training changes model parameters; normal inference uses the already-trained model.
9. Deep learning allows models to learn useful internal representations instead of relying entirely on handcrafted features.
10. Language is difficult because meaning depends heavily on context.

- Coder Army
11. Statistical language models introduced the idea of predicting likely continuations.
 12. RNNs processed language sequentially but struggled with long-range relationships.
 13. Attention allows different parts of a sequence to dynamically use information from relevant parts of the context.
 14. Transformers made attention central to language modelling and became the foundation of modern LLMs.
 15. GPT stands for **Generative Pre-trained Transformer**.
 16. GPT-style language models generate text through repeated **next-token prediction**.
 17. Fluent language does not guarantee factual correctness.
 18. An LLM alone is not a complete production application.
 19. Real AI systems need data, APIs, business rules, security, validation, evaluation and often human oversight.
 20. Building that complete system is where the mindset of a **Forward Deployed Engineer** becomes valuable.